SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)

Assessment Tool Weightage: 40%

Dr. Dumpala Prasanth

QUESTIONS: 10 **Total Marks: 50**

Annexure A

Descriptive Experiment Exam **Duration: 2hrs**

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ε-Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ε-Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different actionselection methods.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Code of Conduct:

I B V Chaitanya Reddy(192425376 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Chaitanya**

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, wellexplained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

EXPERIMENT 1: Installation of Reinforcement Learning Development Environment

Question (Aim):

To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Algorithm:

1. Install Anaconda and Python.

2. Install TensorFlow, Keras, Gymnasium, NumPy, and Matplotlib.

3. Import the required libraries.

4. Verify successful installation.

Code:

```
import tensorflow as tf import
gymnasium as gym import numpy
as np import matplotlib.pyplot as
plt print("TensorFlow:", tf._version
) print("Gymnasium Installed")
print("NumPy:", np._version_)
```

Input & output:

```
Input:
No user input.

Output:
TensorFlow: 2.x.x
Gymnasium Installed
NumPy: 1.x.x
```

Result:

All the required Reinforcement Learning libraries were installed and verified successfully.

EXPERIMENT 2: Familiarization with OpenAI Gym/Gymnasium Environments

Question (Aim):

To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Algorithm:

1. Create the FrozenLake environment.

2. Reset the environment.

3. Select random actions.

4. Observe states and rewards.

5. Stop when the episode ends.

Code: import gymnasium as gym

```
env = gym.make("FrozenLake-v1")
state, _ = env.reset() for i in
range(5):
    action = env.action_space.sample()
    state, reward, done, truncated, info = env.step(action)
    print("State:", state, "Reward:", reward) if done or
    truncated:
        break
env.close()
```

Input & output:

```
EXPERIMENT 2

Input:
No user input.

Output:
State: 0 Reward: 0
State: 4 Reward: 0
State: 5 Reward: 1
(or similar values depending on random actions)
```

Result:

Successfully interacted with the Gymnasium environment and observed states, actions, and rewards.

EXPERIMENT 3: Implementation of the Reinforcement Learning Framework

Question (Aim):

To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Algorithm:

1. Create the environment.

2. Initialize the agent.

3. Select an action.

4. Receive reward.

5. Continue until the episode ends.

Code: import gymnasium as

gym env =

gym.make("CartPole-v1") state,

_ = env.reset() done = False

while not done:

    action = env.action_space.sample()

    state, reward, done, truncated, info = env.step(action)

    print("Reward:", reward) done = done or truncated

env.close()

Input & output:

```
Input:
No user input.

Output:
State: 0 Reward: 0
State: 4 Reward: 0
State: 5 Reward: 1
(or similar values depending on random actions)
```

Result:

The Reinforcement Learning framework was successfully implemented and the agent interacted with the environment.

EXPERIMENT 4: Simulation of a Markov Decision Process (MDP)

Question (Aim):

To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Algorithm:

1. Define states.

2. Assign rewards.

3. Simulate state transitions.

4. Display rewards.

Code:

```
states = ["A", "B", "C"]
rewards = [5, 10, 15] for s, r in
zip(states, rewards):
   print("State:", s, "Reward:", r)
```

Input & output:

```
Input:
No user input.

Output:
State: A Reward: 5
State: B Reward: 10
State: C Reward: 15
```

Result:

The Markov Decision Process was successfully simulated with state transitions and rewards.

EXPERIMENT 5: Bellman Equation Demonstration

Question (Aim):

To implement Bellman Expectation and Bellman Optimality Equations for calculating statevalue and action-value functions and analyze the convergence of value estimation.

Algorithm:

1. Initialize the value function.

2. Apply the Bellman equation.

3. Update the value repeatedly.

4. Display the final value.

Code:

```
gamma    =    0.9

reward = 10

value = 0 for i in

range(10):

    value = reward + gamma * value

print("State Value:", value)
```

Input & output:

```
Input:
Reward = 10
Gamma = 0.9

Output:
State Value: 65.13215599
(Value may vary depending on the number of iterations.)
```

Result:

The Bellman equation successfully estimated the state value.

EXPERIMENT 6: Exploration vs. Exploitation Strategies

Question (Aim):

To implement ε-Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Algorithm:

1. Set epsilon.

2. Generate a random number.

3. Explore if random number is less than epsilon.

4. Otherwise exploit.

Code:

```
import random epsilon

=    0.2    for    i    in

range(10):
```

```
    if random.random() < epsilon:
        print("Explore")
    else:
print("Exploit")
```

Input & output:

```
Input:
Epsilon = 0.2

Output:
Exploit
Explore
Exploit
Exploit
Explore
...
(Output varies because of random action selection.)
```

Result:

The exploration and exploitation strategies were demonstrated successfully.

EXPERIMENT 7: Multi-Armed Bandit Problem Question

(Aim):

To implement the Multi-Armed Bandit problem using ε-Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Algorithm:

1. Create bandits.
2. Select an action.
3. Generate reward.
4. Repeat the process.

Code:

```
import numpy as np
bandits = [0.2, 0.5, 0.8]
for i in range(10):
```

action =

np.random.randint(3)

reward = np.random.binomial(1, bandits[action]) print("Action:",

action, "Reward:", reward)

Input & output:

```
Input:
Bandit Probabilities = [0.2, 0.5, 0.8]

Output:
Action: 2 Reward: 1
Action: 0 Reward: 0
Action: 1 Reward: 1
...
(Output varies for each execution.)
```

Result:

The Multi-Armed Bandit problem was successfully simulated and rewards were generated.

EXPERIMENT 8: Reward Visualization in Reinforcement Learning

Question (Aim):

To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Algorithm:

1. Store rewards.
2. Plot the graph.
3. Display the graph.

Code:

import matplotlib.pyplot as plt

rewards =

[1,2,3,4,5,6,7,8,9,10]

plt.plot(rewards)

plt.xlabel("Episode")

plt.ylabel("Reward")

plt.title("Reward vs Episode")

plt.show()

Input & output:

```
Input:
Rewards = [1,2,3,4,5,6,7,8,9,10]

Output:
A line graph showing Reward vs Episode is displayed.
```

Result:

The reward graph was successfully displayed using Matplotlib.

EXPERIMENT 9: TensorFlow and Keras-Based Neural Network for Reinforcement Learning

Question (Aim):

To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Algorithm:

1. Create a Sequential model.

2. Add Dense layers.

3. Compile the model.

4. Display the model summary.

Code:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
    Dense(24, activation='relu', input_shape=(4,)),
    Dense(24, activation='relu'),
    Dense(2)
])
model.compile(optimizer='adam', loss='mse')
model.summary()
```

Input & output:

```
Input:
Input Shape = (4,)
Hidden Layers = 24, 24
Output Layer = 2

Output:
Model: "sequential"
Dense (24)
Dense (24)
Dense (2)
Total Parameters: ...
(Model summary is displayed.)
```

Result:

The feed-forward neural network was successfully created and compiled.

EXPERIMENT 10: Mini Reinforcement Learning Project Using CartPole Question

(Aim):

To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Algorithm:

1. Create the CartPole environment.
2. Reset the environment.
3. Select random actions.
4. Calculate the total reward.
5. Display the final reward.

Code: import gymnasium as

gym env =

gym.make("CartPole-v1") state,

_ = env.reset() total_reward = 0

done = False while not done:

action = env.action_space.sample()

state, reward, done, truncated, info = env.step(action)

total_reward += reward done

= done or truncated

print("Total Reward:", total_reward) env.close()

Input & output:

```
Input:
No user input.

Output:
Total Reward: 25.0
(or any value depending on the episode performance.)
```

Result:

The CartPole Reinforcement Learning agent successfully interacted with the environment, and the total episode reward was obtained.